

Open in app ↗

**Medium**

Search



Write

Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)

Implementing YoloV8 in detail for beginners.



Nandini Lokesh Reddy

Follow

6 min read · Mar 11, 2024



99



Understanding the intricacies of YOLOv8 from research papers is one aspect, but translating that knowledge into practical implementation can often be a different journey altogether. YOLO, standing for “You Only Look Once,” has gained fame for its versatility in tasks beyond mere object detection. While it’s widely known for its prowess in detection, YOLO is also capable of classification, tracking, and segmenting objects within images or video frames.

In this blog series, we’ll delve into the practical aspects of implementing YOLO from scratch. We’ll start by understanding the core principles of YOLO and its architecture, as outlined in the research papers. From there, we’ll transition into hands-on implementation, exploring how to build and train YOLO models for various tasks.

So, what is YOLO?

In the realm of computer vision and object detection, YOLO stands out as a revolutionary approach that has redefined the landscape of real-time object detection. YOLO, an acronym for “You Only Look Once,” is a deep learning-based algorithm designed to detect objects in images or video frames swiftly and accurately.

Traditional object detection methods typically involve multiple stages, such as region proposal, feature extraction, and classification, which can be computationally expensive and time-consuming. YOLO, however, takes a fundamentally different approach by formulating object detection as a single regression problem, enabling it to achieve remarkable speed without compromising on accuracy.

Key Features of YOLO:

1. Unified Framework: Unlike traditional methods that rely on separate stages for object localization and classification, YOLO unifies these tasks into a single neural network model. This end-to-end approach allows YOLO to simultaneously predict bounding boxes and class probabilities for multiple objects in a single pass through the network.

2. Real-Time Performance: YOLO’s unified architecture and efficient design enable it to achieve remarkable speed, making it suitable for real-time applications such as autonomous driving, video surveillance, and augmented reality. With YOLO, object detection can be performed at frame rates exceeding 30 frames per second, even on resource-constrained devices.

3. High Accuracy: Despite its speed, YOLO maintains competitive accuracy compared to traditional multi-stage approaches. By leveraging deep

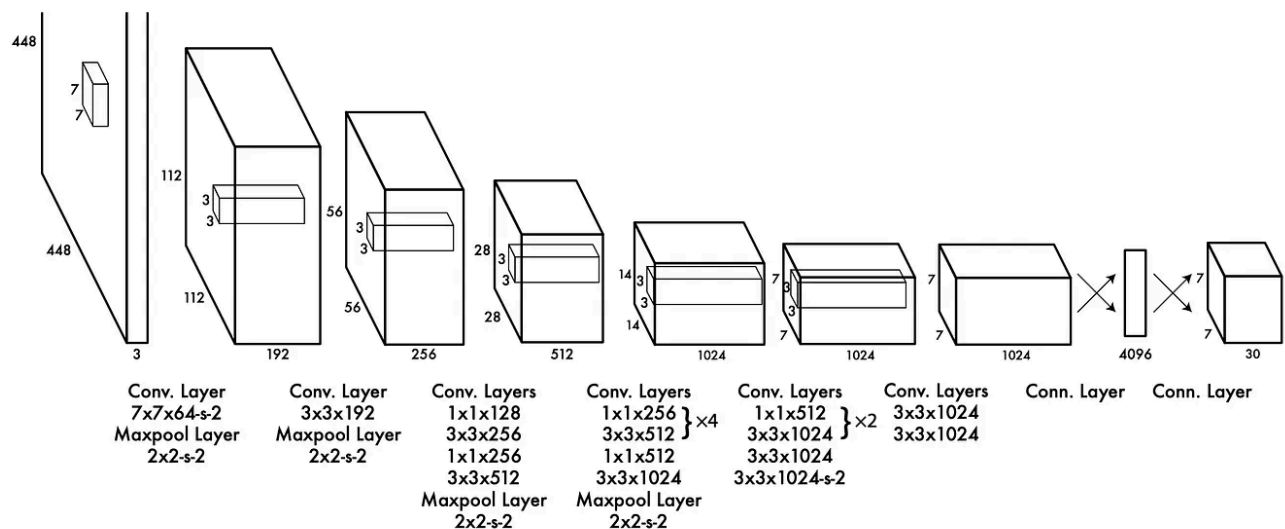
convolutional neural networks (CNNs) trained on large-scale datasets like COCO (Common Objects in Context), YOLO is capable of detecting a wide range of objects with high precision and recall.

4. Robustness to Scale and Aspect Ratio: YOLO is designed to handle objects of various scales and aspect ratios effectively. Through the use of a grid-based approach and anchor boxes, YOLO can detect objects at different positions and scales within an image, making it robust to variations in object size and orientation.

5. Flexibility and Adaptability: YOLO is highly flexible and can be customized to suit different application domains and datasets. Researchers and developers can fine-tune YOLO models on specific datasets or modify the architecture to meet the requirements of specialized tasks, such as pedestrian detection, vehicle detection, or medical image analysis.

This is the base paper of YOLO: <https://arxiv.org/abs/1506.02640>

YOLO Architecture:



1. **Input Image:** The YOLO algorithm takes an input image of fixed size. This image is divided into a grid of cells, typically with a size of $S \times S$.
2. **Convolutional Neural Network (CNN):** YOLO utilizes a deep convolutional neural network as its backbone to extract features from the input image. The architecture of the CNN consists of multiple convolutional layers followed by max-pooling layers, which progressively reduce the spatial dimensions of the input while increasing the depth of feature maps.
3. **Fully Connected Layers:** Towards the end of the network, YOLO incorporates fully connected layers, also known as dense layers, to process the extracted features and generate predictions.
4. **Grid Division:** The input image grid is divided into $S \times S$ cells. Each cell is responsible for predicting a fixed number of bounding boxes and their corresponding confidence scores and class probabilities.
5. **Bounding Box Prediction:** For each grid cell, YOLO predicts a fixed number of bounding boxes (typically B). Each bounding box is characterized by five attributes: $(x, y, w, h, \text{confidence})$, where (x, y) represents the center coordinates of the box relative to the grid cell, (w, h) represent the width and height of the box relative to the entire image, and confidence represents the confidence score that the box contains an object and the accuracy of the bounding box.
6. **Class Prediction:** In addition to bounding boxes, each grid cell predicts the probability distribution over predefined classes. YOLO uses softmax activation to compute the class probabilities, indicating the likelihood of each class being present in the bounding box.
7. **Output Prediction:** The final output of the YOLO model is a tensor of shape $(S, S, (B * 5 + C))$, where B is the number of bounding boxes per cell, 5 corresponds to the bounding box attributes $(x, y, w, h, \text{confidence})$, and C is the number of classes.

By jointly predicting bounding boxes and class probabilities across the entire image in a single forward pass through the network, YOLO achieves remarkable speed and efficiency while maintaining competitive accuracy in object detection tasks.

Now, let's begin with implementation!



Step-1: Install the Yolo model from Ultralytics; if you already have installed ignore this step;

```
#installing Ultralytics
%pip install ultralytics

import ultralytics
#to check which version
ultralytics.checks()

#importing YOLO from ultralytics
from ultralytics import YOLO
```

or directly clone from GitHub

```
git clone https://github.com/ultralytics/ultralytics.git
```

Or you can directly use it from CLI (Command Line Interface)

```
!yolo task=detect \ mode=predict \ model=yolov8n.pt \ source="image.jpg"
```

- The task can be {detect, segment, classify}
- whereas mode can be {train, predict, val, export}
- model as an uninitialized .yaml or as a previously trained .pt file
- Source as the path/to/data

Step 2 depends on whether you need to train the Yolo based on your dataset or you need the generalized version of Yolo.

Step-2: Generalized Version of Yolo-v8: This is where you just run the pre-trained model and get your desired results.

```
model = YOLO("Path/to/.pt file")  
  
#for detection.  
result = model.predict("path/to/image-or-video")
```

You have to download the weights file depending on the task you are choosing and model accuracy:

```
!yolo predict model=yolov8n.pt source='https://ultralytics.com/images/zidane.jpg'
```

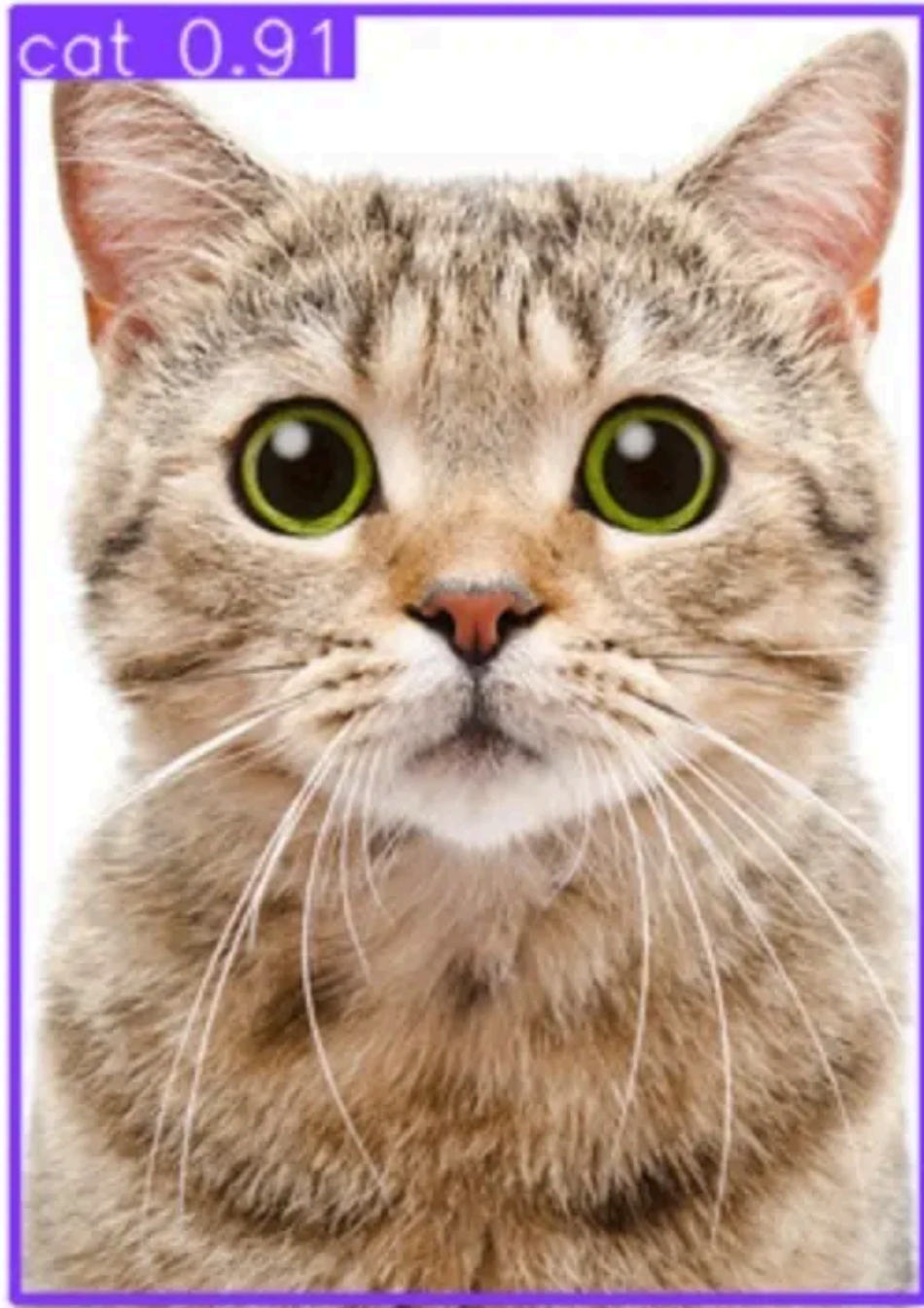
After executing the code provided above on an image, you will observe an output similar to this:

```
image 1/1 /content/cat.jpeg: 448x640 1 cat, 109.4ms  
Speed: 4.7ms preprocess, 109.4ms inference, 531.6ms postprocess per image at sha
```

Example:



input image



Output image

In summary, YOLO represents a paradigm shift in object detection, offering a potent combination of speed, accuracy, and versatility. Its ability to perform real-time object detection with high precision has propelled its adoption across a wide range of industries and applications, heralding a new era in computer vision research and technology.

Conclusion:

In this post, we've taken the first steps towards understanding and implementing YOLO (You Only Look Once) for object detection tasks. We've covered the basics, from introducing the concept of YOLO and its architecture to implementing simple detection on a cat image using a pre-trained model. By doing so, we've gained insights into how YOLO operates and its potential applications.

Looking ahead, we'll continue our exploration by diving deeper into customizing YOLO for specific tasks or datasets. We'll explore techniques for fine-tuning pre-trained models, adapting YOLO to new datasets, and optimizing performance for different applications. Whether you're interested in enhancing object detection accuracy, tackling new challenges, or exploring advanced YOLO features, the next post will provide valuable insights and practical guidance.

Stay tuned for the next installment, where we'll unlock the full potential of YOLO and discover how to tailor it to your specific needs.

If you have any questions or topics you'd like to see covered in future posts, feel free to share them in the comments below.

Yolov8

Computer Vision

Yolov8 Implementation

Object Detection

**Written by Nandini Lokesh Reddy**

Follow